socmint-narrative-tracker

Git Repository & Setup

Run in terminal to clone, verify structure, and launch the app:

```
git clone https://github.com/cchristion/socmint-narrative-tracker
cd socmint-narrative-tracker
tree -a -I '.git*|uv.lock|.py*|py*'
uv sync
uv run streamlit run app.py
```

- **Modular Package (`src/socmint_narrative_tracker/`):** `collectors.py`, `narratives.py`, `origin.py`, `coordination.py`, `exporter.py`
- **Data & Artifacts:** Benchmark (`data/synthetic_posts.json`), deduplicated live cache (`data/cache/`), exports (`output/posts.csv`, `intelligence_report.json`, `report.html`)

Deliverable 1: Module in Action (Via App)

In the Streamlit Sidebar (`http://localhost:8501`):

1. **Topic:** `Acme Bank` | **Dates:** `2026-10-25` to `Today`
2. **Platforms:** `Telegram`, `X`, `YouTube`
3. **Data Source:** `Synthetic Benchmark (data/synthetic_posts.json)`
4. Click 🚀 **Run Intelligence Pipeline**

Walkthrough in App:

- **1. Main Narratives:** View hourly timeline & per-platform charts; inspect the 4 separated narratives (`6, 5, 2, 1` posts) and `PASSED (100% Grounded)` status.
- **2. Coordinated Account Rings:** Expand `COORD-RING-01` & `COORD-RING-02` to show confidence scores and all 5 triggered heuristics.
- **3. Patient Zero & Spread:** Show `@shadow_fin_alert` (Telegram `+0m` → X `+2m` → YouTube `+45m`).
- **4. Exports:** Click download buttons for `CSV`, `JSON`, and `HTML` reports.

Deliverable 2: Live Test Inputs (Via App)

In the Streamlit Sidebar:

1. **Topic:** Enter live test input provided during demo (e.g., `Bitcoin`)
2. **Data Source:** Select `Live OSINT Scrapers + Cache (Telegram, X, YouTube)`
3. *(Optional)* Check 🔄 **Force Live Refresh (Ignore Cache)** to scrape fresh data
4. Click 🚀 **Run Intelligence Pipeline**

Walkthrough in App:

- **Live Telemetry & Cache:** Combines and deduplicates live scrapes with `data/cache/<topic>.json`.
- **Interactive Threshold Tuning:** Adjust **Coordination Text Similarity** (`0.70–0.98`) and **Narrative Clustering Distance** (`0.15–0.60`) sliders live.
- **Verify Live Links & HTML Report:** Click live source URLs in Section 4 table and open `output/report.html` in browser.

Deliverable 3 & Explain 1: Methodology & Design

- **Unified Schema (`collectors.py`):** Maps Telegram, X, and YouTube into 12 standardized fields. Zero synthetic strings in Python code; unauthenticated snippets without views store `None` for `engagement` instead of fake math.
- **Unsupervised Clustering (`narratives.py`):** Embeds text with `all-MiniLM-L6-v2` (384-d vectors) + Cosine `AgglomerativeClustering` (`distance_threshold=0.25`) so cluster count K is discovered automatically. Strips target query words from `TfidfVectorizer` stop-words.

- **Patient Zero & Lag (`origin.py`):** Sorts clusters chronologically (`timestamp_dt` ascending) to isolate `iloc[0]` and calculates cross-platform lag (`+Xm`).
- **Multi-Signal Graph Attribution (`coordination.py`):** Models accounts in `networkx.Graph` across 5 rules: (1) Cosine sim ≥ 0.85, (2) Temporal sync $\leq 300\text{s}$, (3) Batch creation $\leq 24\text{h}$, (4) Shared MD5 `avatar_hash`, (5) Shared `amplified_handle`.

Explain 2 & 3: Tools Used & AI Verification

Tools, Libraries & Endpoints Used:

- **Collection:** `scrapetube` (YouTube innertube API), `ddgs` + `requests` + `BeautifulSoup4` (Telegram `t.me/s/` public web previews & X discovery).
- **NLP & Graph Math:** `sentence-transformers` (`all-MiniLM-L6-v2`), `scikit-learn` (`AgglomerativeClustering`, `TfidfVectorizer`), `networkx` (`connected_components`).
- **Dashboard & Reporting:** `streamlit`, `plotly`, `pandas` (`CSV`, `JSON`, `HTML`).

How AI Was Used & Checked:

- **Deterministic Embeddings:** Used local `all-MiniLM-L6-v2` cosine distances and graph math rather than generative LLMs to prevent non-deterministic clustering or hallucinated bot rings.
- **Automated Grounding Check (`ai_grounding_check`):** Verifies every extracted TF-IDF narrative keyword appears verbatim in the lowercase cluster text (`PASSED (100% Grounded)`).

Explain 4, 5 & 6: Rate Limits, Accuracy & Limits

- **Rate Limits, Blocking & Platform Restrictions:**
  - Uses zero-login public endpoints (`scrapetube` , Telegram `t.me/s/` newest message `[-1]` , `ddgs`) to avoid login walls, asyncio session bans, or paid API quotas.
  - Caches and deduplicates live results by `post_id` / `link` in `data/cache/<slug>.json` with a **Force Live Refresh** option to prevent HTTP 429 blocks.

- **Accuracy (False Positives & False Negatives Found):**
  - **Clustering FN (Fixed):** At `distance_threshold=0.55`, all posts sharing `"Acme Bank"` merged into 1 cluster. Tightening `0.55` $\rightarrow$ `0.35` $\rightarrow$ `0.25` and stripping query stop-words cleanly separated all 4 sub-narratives.
  - **Coordination FP (Fixed):** Requires ≥ 2 **distinct heuristics** (or cosine similarity ≥ 0.92) to form a graph edge, preventing organic viral news sharing from triggering botnet alerts.
  - **Coordination FN (Trade-off):** Heavily paraphrased campaigns (< 0.65 similarity) staggered $> 30\text{m}$ evade default cutoffs unless lowered via the sidebar sliders.

- **Limitations & Assumptions:**
  - Public posts only (private Telegram groups require authenticated MTProto sessions); unauthenticated web snippets lack live `author_created_at`, so live mode relies on text similarity, temporal sync, and amplification while benchmark mode tests all 5 heuristics.

Thank You